Project : Census Income

1. Perform Exploratory Data Analysis to find key insights.

Code:-

```
# Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt



# Display basic information
print("Dataset Shape:", data.shape)  # Number of rows and columns
print("Columns:\n", data.columns)  # Column names
print(data.head())  # First few rows
print(data.info())  # Data types and missing values

# Check for missing values
print("\nMissing Values:\n", data.isnull().sum())

# Strip whitespace from column names (just in case)
data.columns = data.columns.str.strip()

# Ensure the target column exists (rename if necessary)
if 'income' not in data.columns:
    print("Target column not found. Please check column names!")
else:
    # Target variable distribution
    sns.countplot(data['income'])
    plt.title('Income Distribution')
    plt.show()

# Plot distributions for numerical columns
numerical_cols = data.select_dtypes(include=['int64', 'float64']).columns
for col in numerical_cols:
    sns.histplot(data[col], kde=True, bins=30)
    plt.title(f'Distribution of {col}')
    plt.show()

# Plot bar charts for categorical columns
categorical_cols = data.select_dtypes(include=['object']).columns
for col in categorical_cols:
    sns.countplot(y=data[col])
    plt.title(f'Distribution of {col}')
    plt.show()

# Check correlation for numerical columns
if len(numerical_cols) > 1:
    sns.heatmap(data[numerical_cols].corr(), annot=True, cmap='coolwarm')
```

```
    plt.title("Correlation Heatmap")
    plt.show()

# Compare categorical columns with target
if 'income' in data.columns:
    for col in categorical_cols:
        if col != 'income':  # Skip the target column
            sns.countplot(data[col], hue=data['income'])
            plt.title(f'{col} vs Income')
            plt.show()
```

Ans:-

```
Dataset Shape: (32561, 15)
Columns:
 Index(['age', 'workclass', 'fnlwgt', 'education', 'education-num',
       'marital-status', 'occupation', 'relationship', 'race', 'sex',
       'capital-gain', 'capital-loss', 'hours-per-week', 'native-country',
       'annual_income'],
      dtype='object')
   age         workclass  fnlwgt  education  education-num  \
0   39         State-gov   77516  Bachelors             13
1   50  Self-emp-not-inc   83311  Bachelors             13
2   38           Private  215646    HS-grad              9
3   53           Private  234721       11th              7
4   28           Private  338409  Bachelors             13

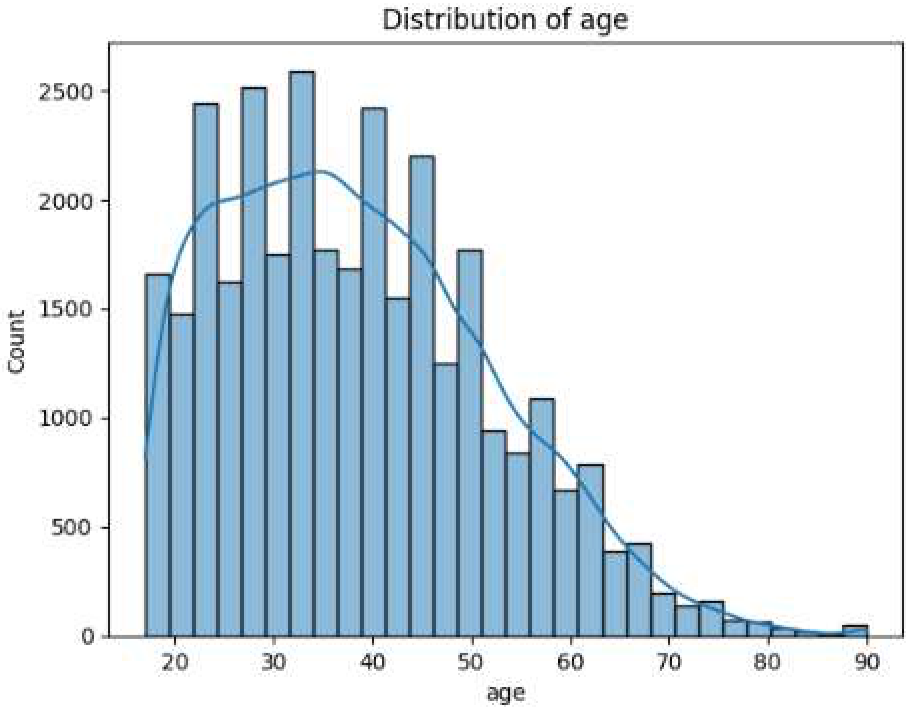
       marital-status         occupation   relationship   race     sex  \
0       Never-married       Adm-clerical  Not-in-family  White    Male
1  Married-civ-spouse    Exec-managerial        Husband  White    Male
2            Divorced  Handlers-cleaners  Not-in-family  White    Male
3  Married-civ-spouse  Handlers-cleaners        Husband  Black    Male
4  Married-civ-spouse     Prof-specialty           Wife  Black  Female

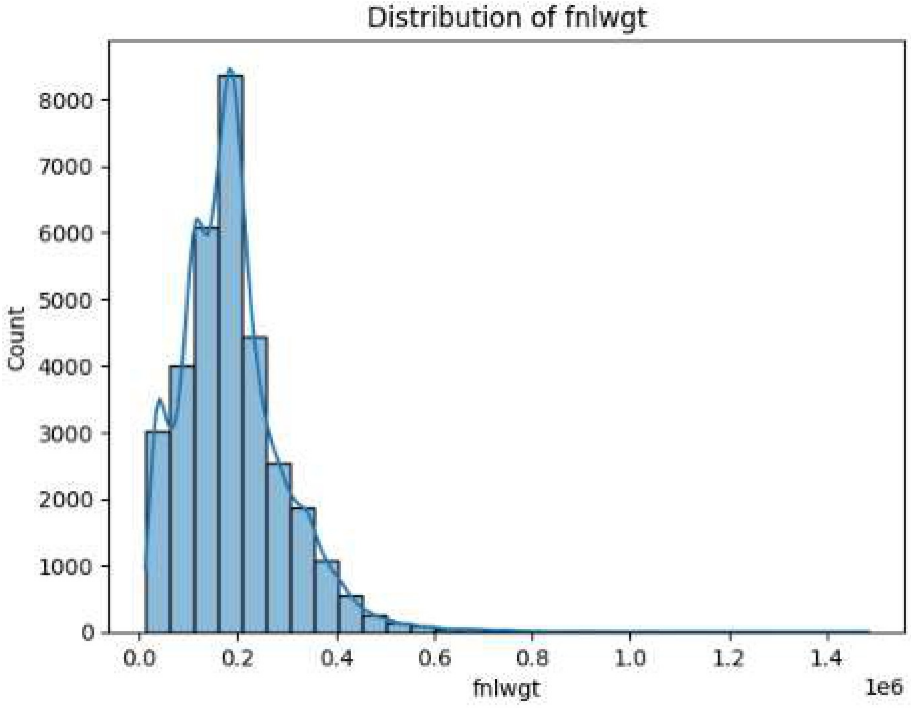
   capital-gain  capital-loss  hours-per-week native-country annual_income
0          2174             0              40  United-States         <=50K
1             0             0              13  United-States         <=50K
2             0             0              40  United-States         <=50K
3             0             0              40  United-States         <=50K
4             0             0              40           Cuba         <=50K
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32561 non-null  int64
 1   workclass       32561 non-null  object
 2   fnlwgt          32561 non-null  int64
 3   education       32561 non-null  object
 4   education-num   32561 non-null  int64
 5   marital-status  32561 non-null  object
 6   occupation      32561 non-null  object
 7   relationship    32561 non-null  object

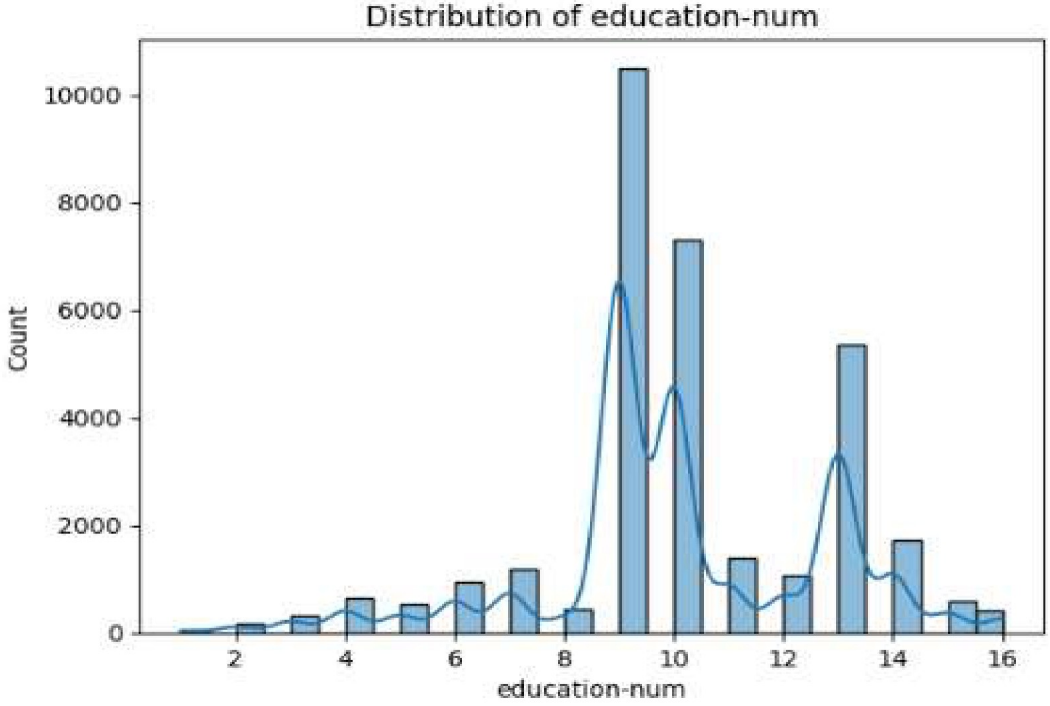
 8   race            32561 non-null  object
 9   sex             32561 non-null  object
 10  capital-gain    32561 non-null  int64
 11  capital-loss    32561 non-null  int64
 12  hours-per-week  32561 non-null  int64
 13  native-country  32561 non-null  object
 14  annual_income   32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
None

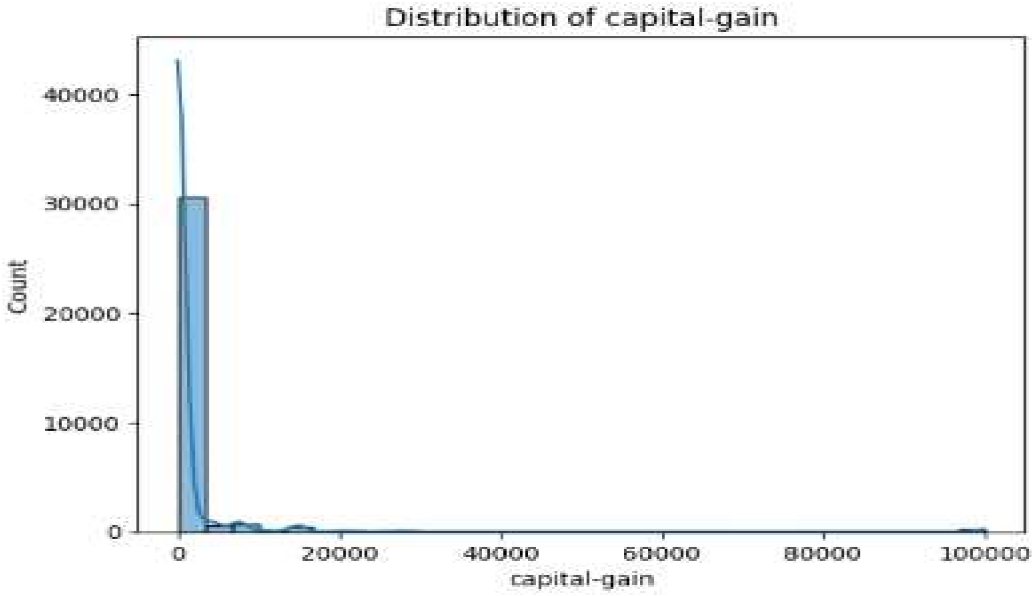
Missing Values:
 age               0
workclass          0
fnlwgt             0
education          0
education-num      0
marital-status     0
occupation         0
relationship       0
race               0
sex                0
capital-gain       0
capital-loss       0
hours-per-week     0
native-country     0

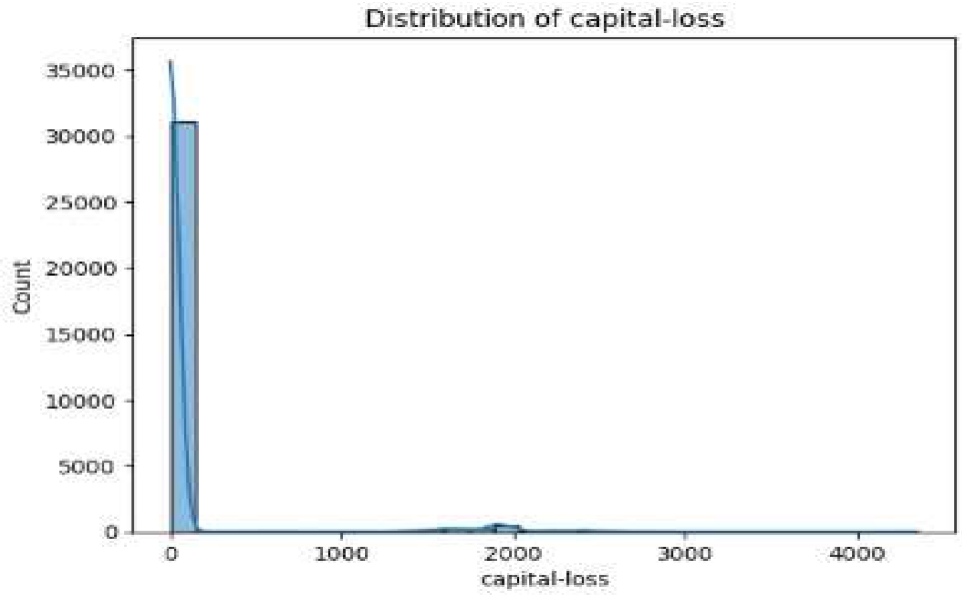
annual_income      0
dtype: int64
Target column not found. Please check column names!
```

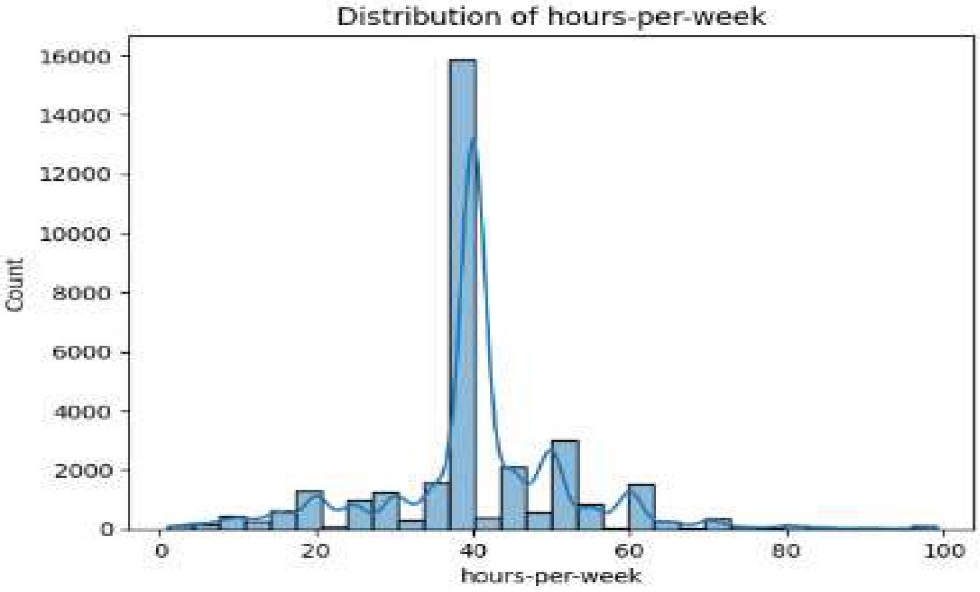

Distribution of age
Count
2500
2000
1500
1000
500
0
20
30
40
50
60
70
80
90
age

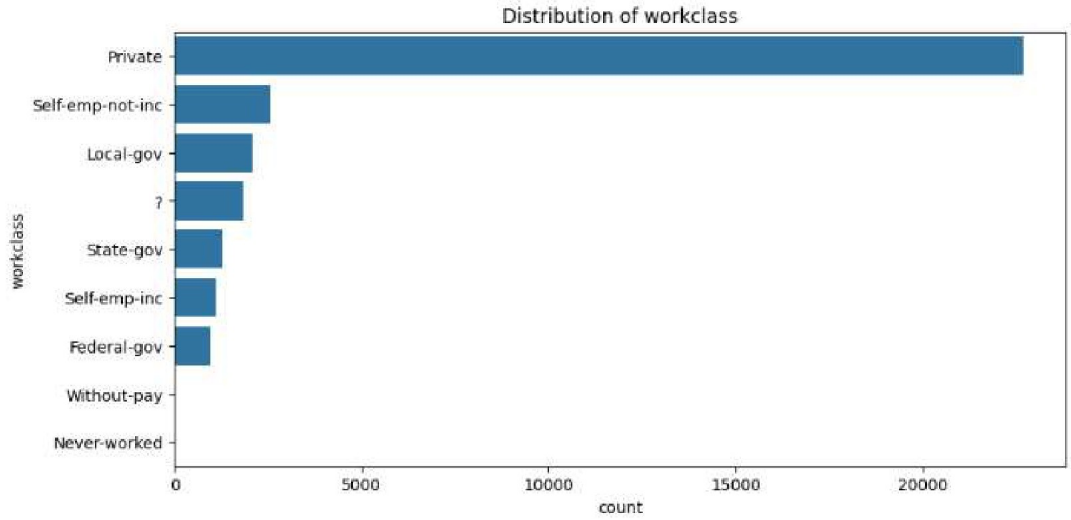

Distribution of fnlwgt
Count
8000
7000
6000
5000
4000
3000
2000
1000
0
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
fnlwgt
1e6

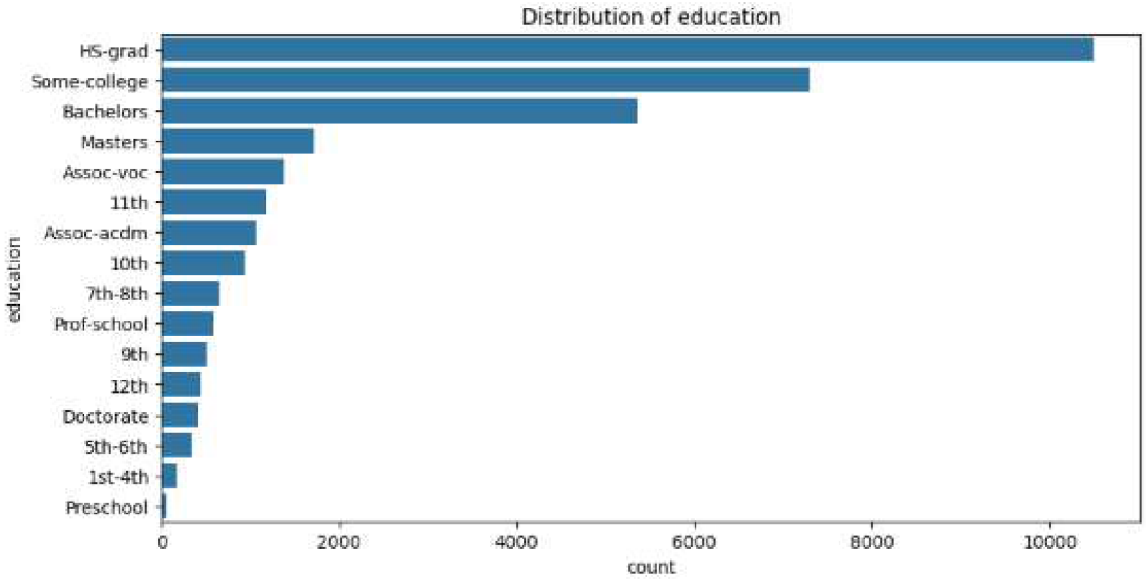

Distribution of education-num
Count
10000
8000
6000
4000
2000
0
2
4
6
8
10
12
14
16
education-num

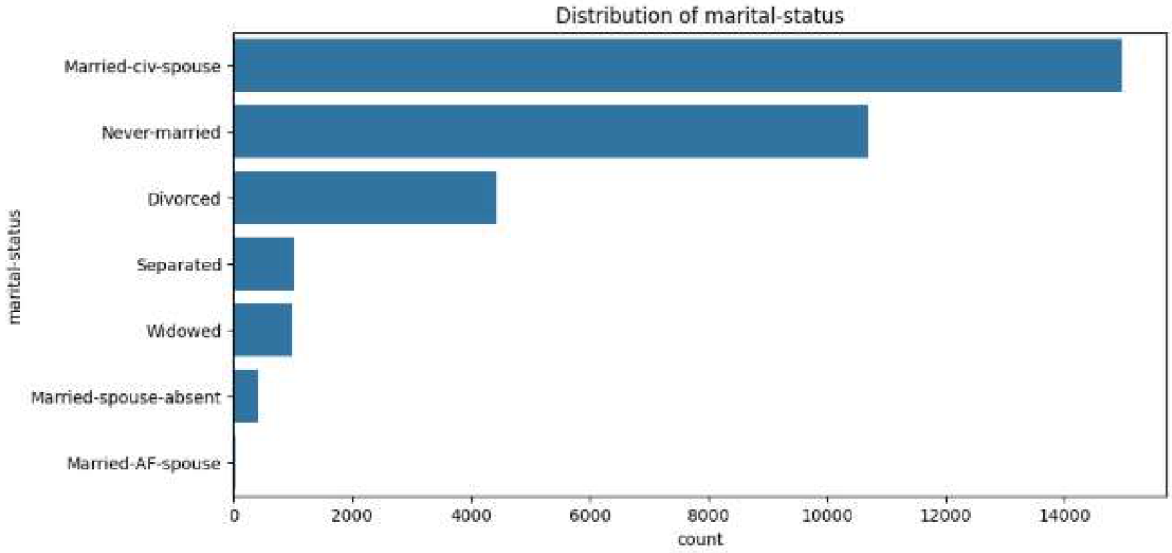

Distribution of capital-gain
Count
40000
30000
20000
10000
0
0
20000
40000
60000
80000
100000
capital-gain

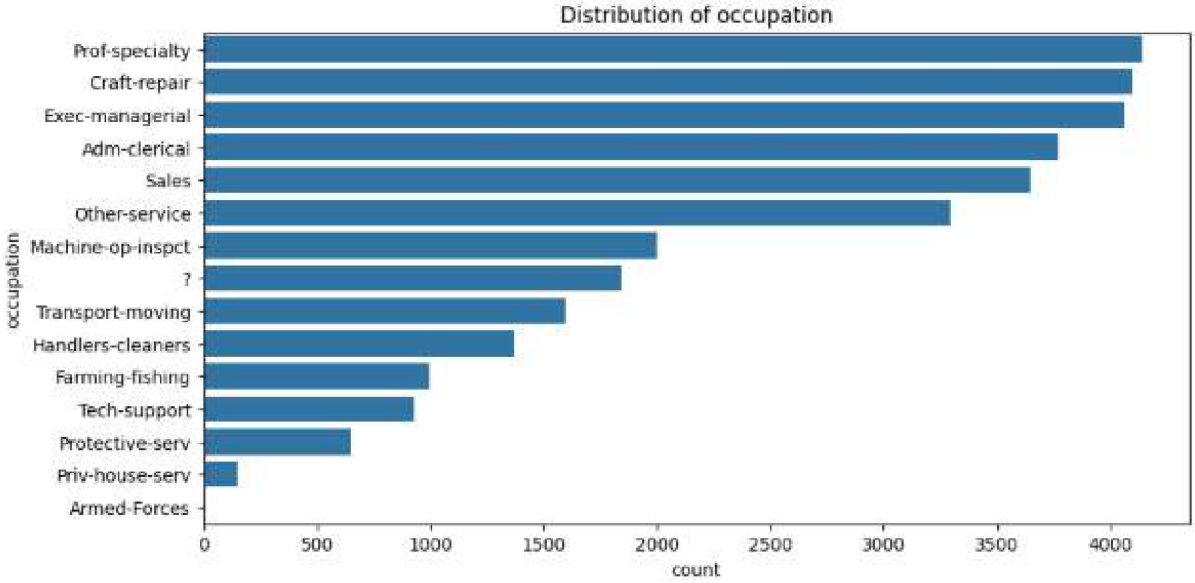

Distribution of capital-loss
35000
30000
25000
20000
15000
10000
5000
0
Count
0
1000
2000
3000
4000
capital-loss

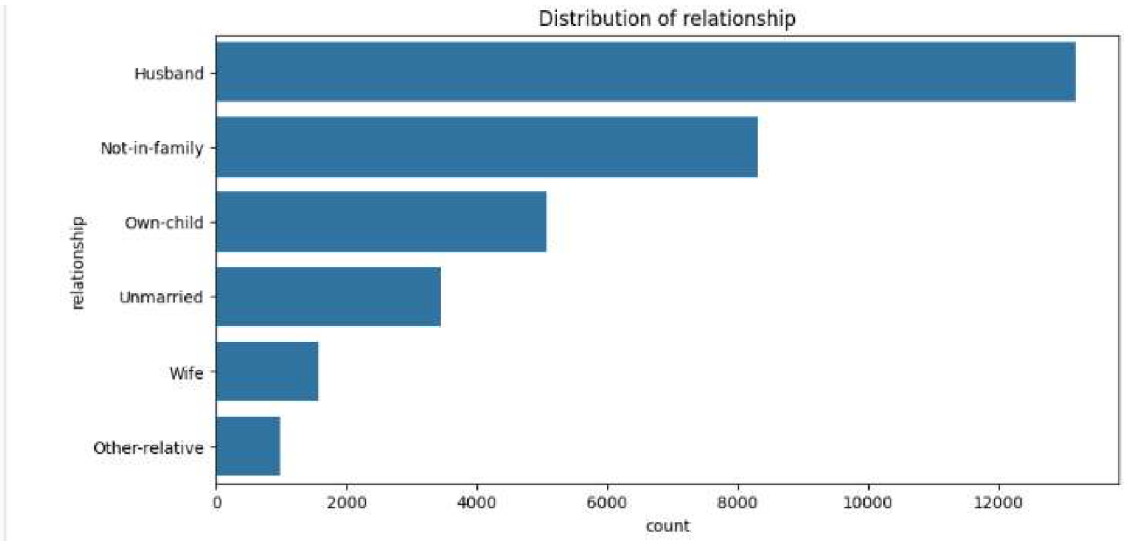

Distribution of hours-per-week
16000
14000
12000
10000
8000
6000
4000
2000
0
Count
0
20
40
60
80
100
hours-per-week

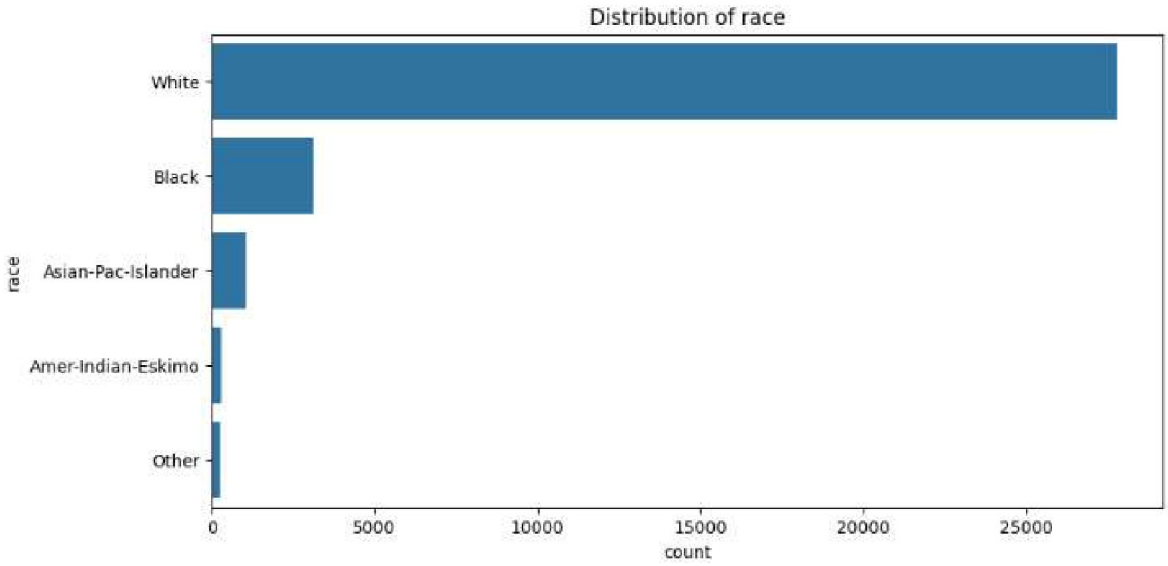

Distribution of workclass
Private
Self-emp-not-inc
Local-gov
?
State-gov
Self-emp-inc
Federal-gov
Without-pay
Never-worked
workclass
0
5000
10000
15000
20000
count

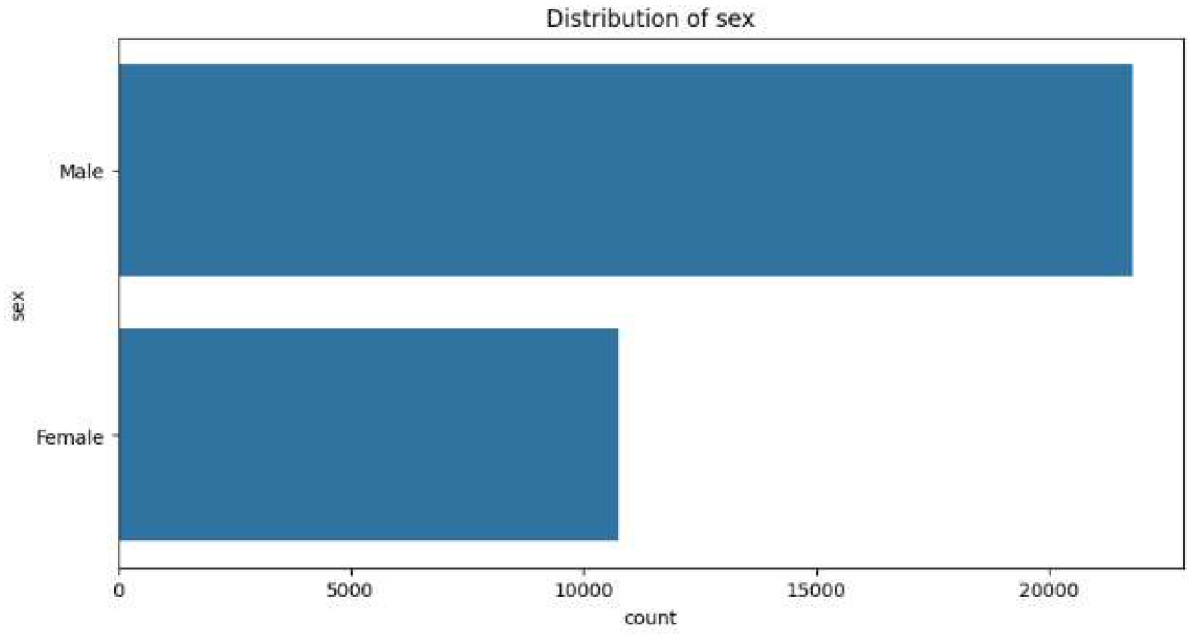

Distribution of education
HS-grad
Some-college
Bachelors
Masters
Assoc-voc
11th
Assoc-acdm
10th
7th-8th
Prof-school
9th
12th
Doctorate
5th-6th
1st-4th
Preschool
education
0
2000
4000
6000
8000
10000
count

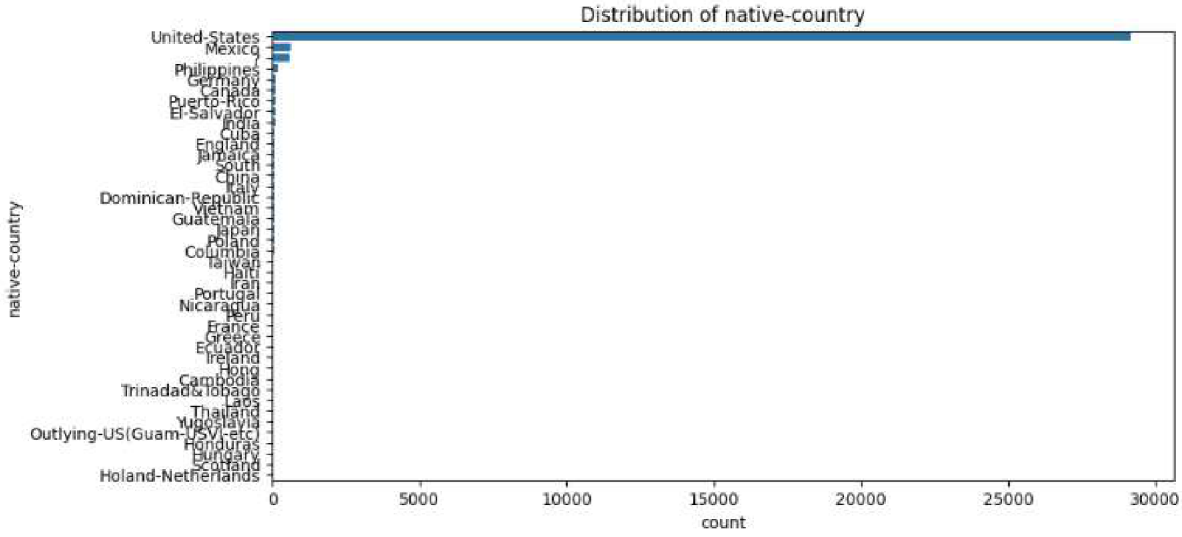

Distribution of marital-status
Married-civ-spouse
Never-married
Divorced
Separated
Widowed
Married-spouse-absent
Married-AF-spouse
marital-status
0
2000
4000
6000
8000
10000
12000
14000
count

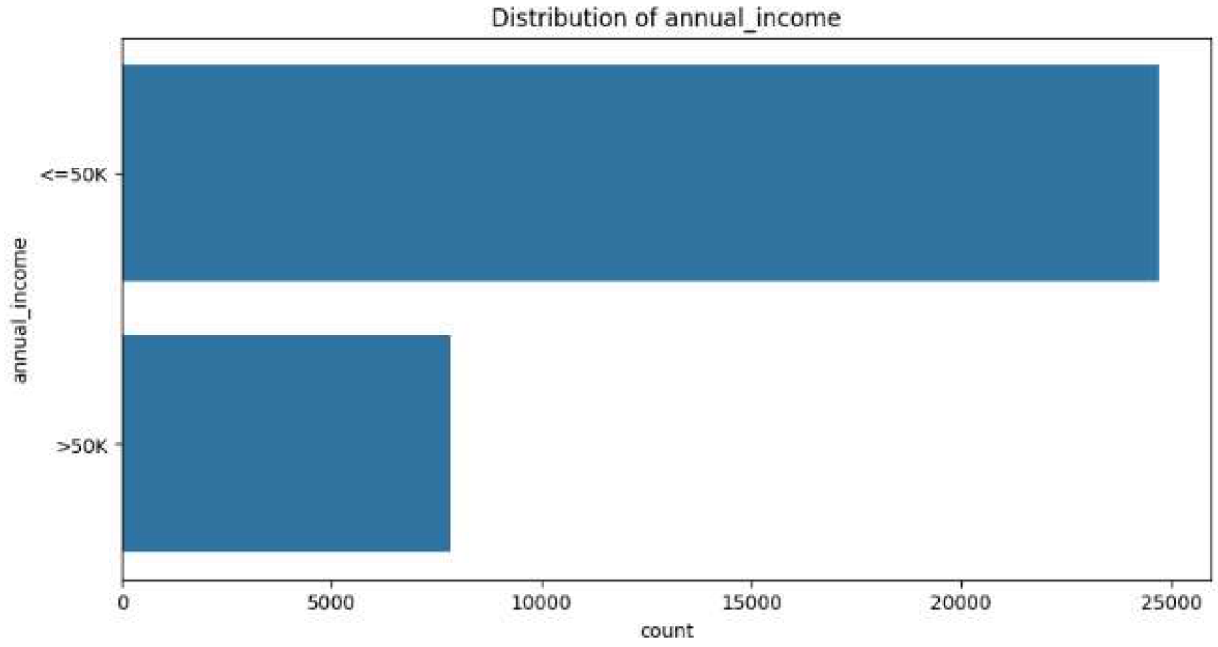

Distribution of occupation
Prof-specialty
Craft-repair
Exec-managerial
Adm-clerical
Sales
Other-service
Machine-op-inspct
?
Transport-moving
Handlers-cleaners
Farming-fishing
Tech-support
Protective-serv
Priv-house-serv
Armed-Forces
occupation
0
500
1000
1500
2000
2500
3000
3500
4000
count

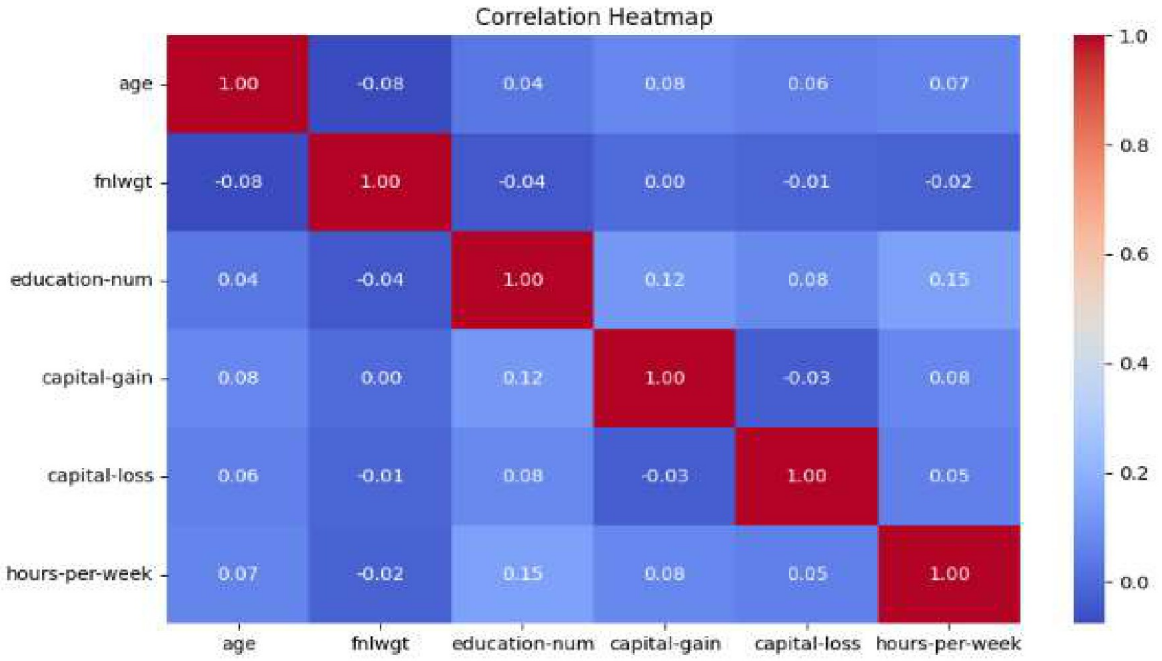

Distribution of relationship
Husband
Not-in-family
Own-child
Unmarried
Wife
Other-relative
relationship
0
2000
4000
6000
8000
10000
12000
count

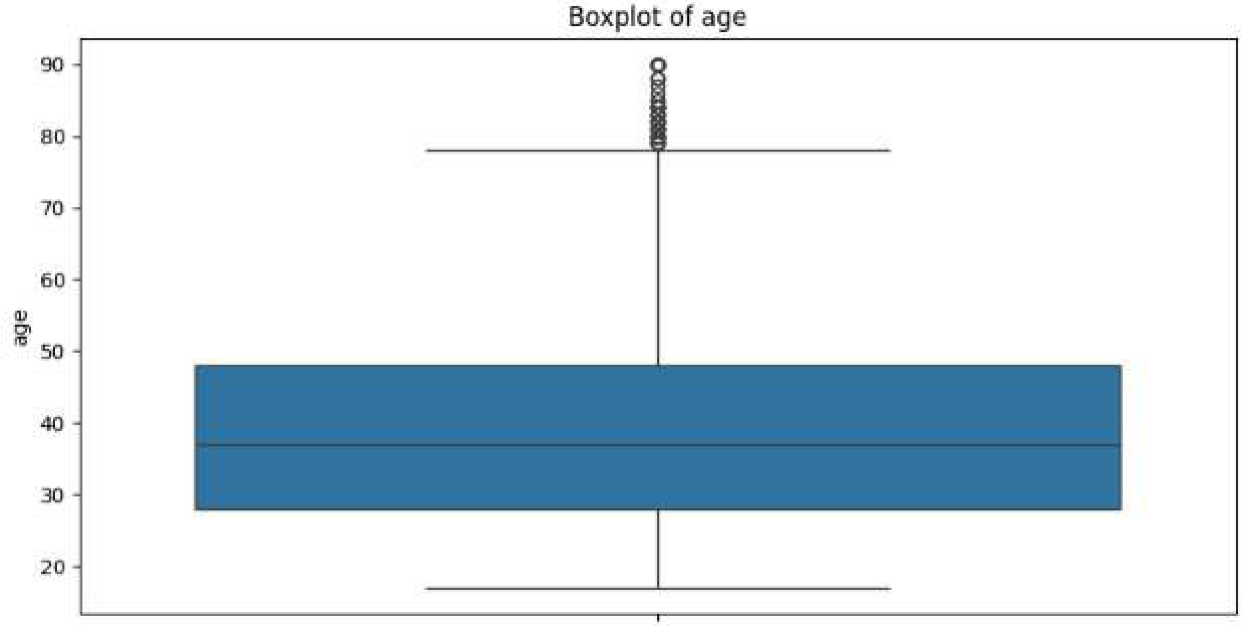

Distribution of race
White
Black
Asian-Pac-Islander
Amer-Indian-Eskimo
Other
race
0
5000
10000
15000
20000
25000
count

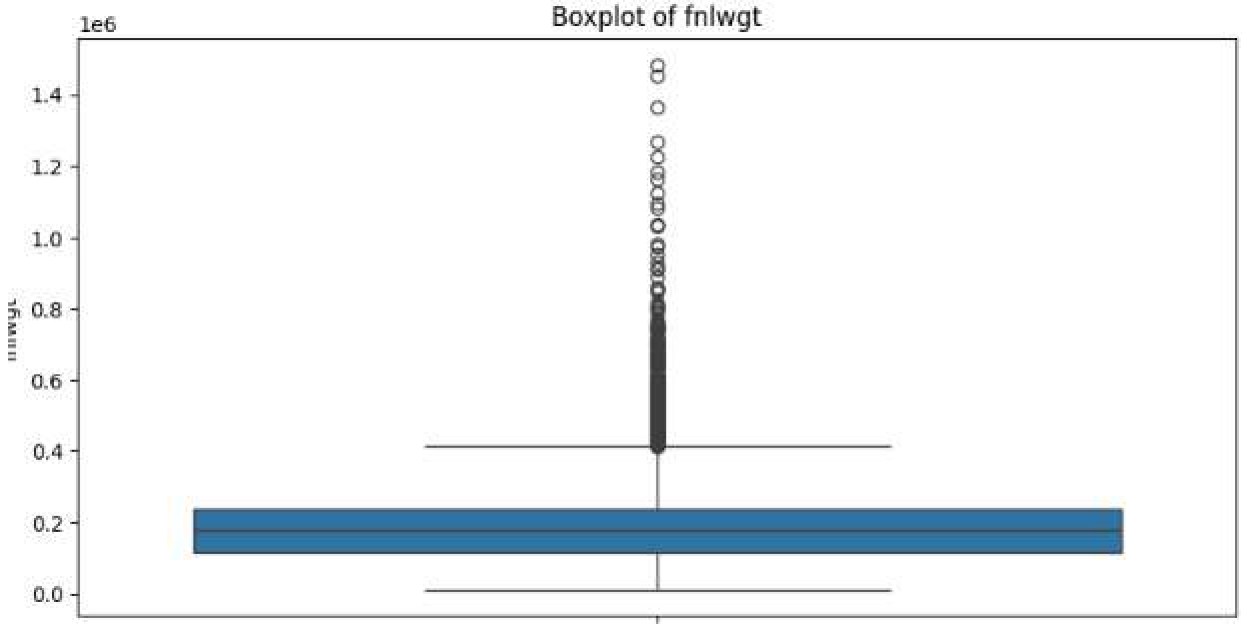

Distribution of sex
Male
Female
sex
0
5000
10000
15000
20000
count

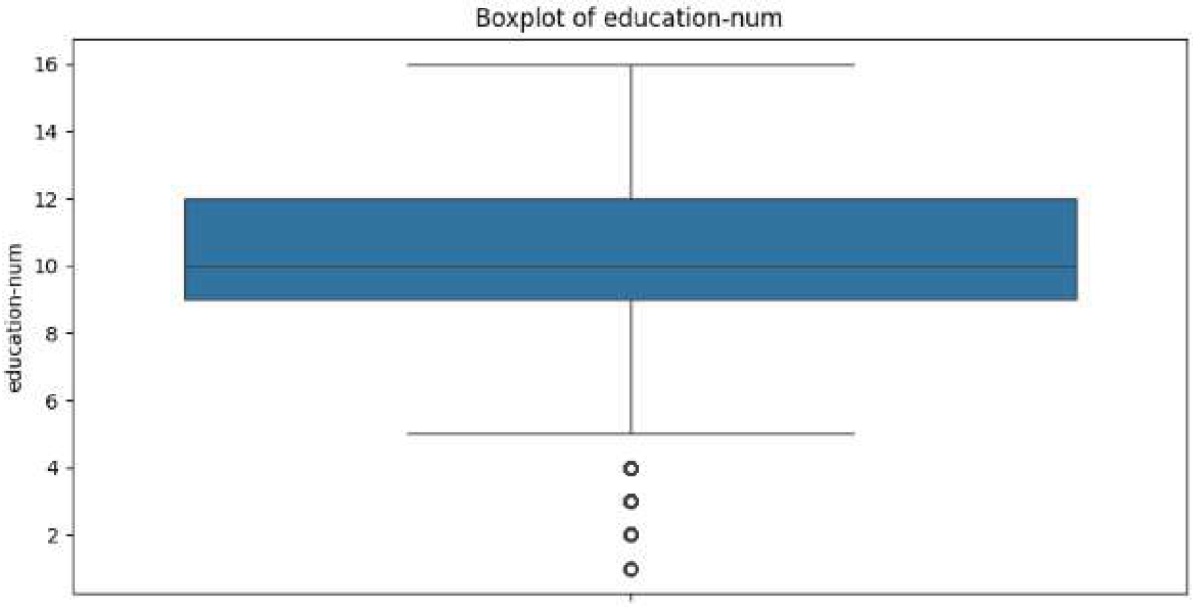

Distribution of native-country
United-States
Mexico
Philippines
Germany
Canada
Puerto-Rico
El-Salvador
India
Cuba
England
Jamaica
South
China
Italy
Dominican-Republic
Vietnam
Guatemala
Japan
Poland
Columbia
Taiwan
Haiti
Iran
Portugal
Nicaragua
Peru
France
Greece
Ecuador
Ireland
Hong
Cambodia
Trinadad&Tobago
Laos
Thailand
Yugoslavia
Outlying-US(Guam-USVI-etc)
Honduras
Hungary
Scotland
Holand-Netherlands
native-country
0
5000
10000
15000
20000
25000
30000
count

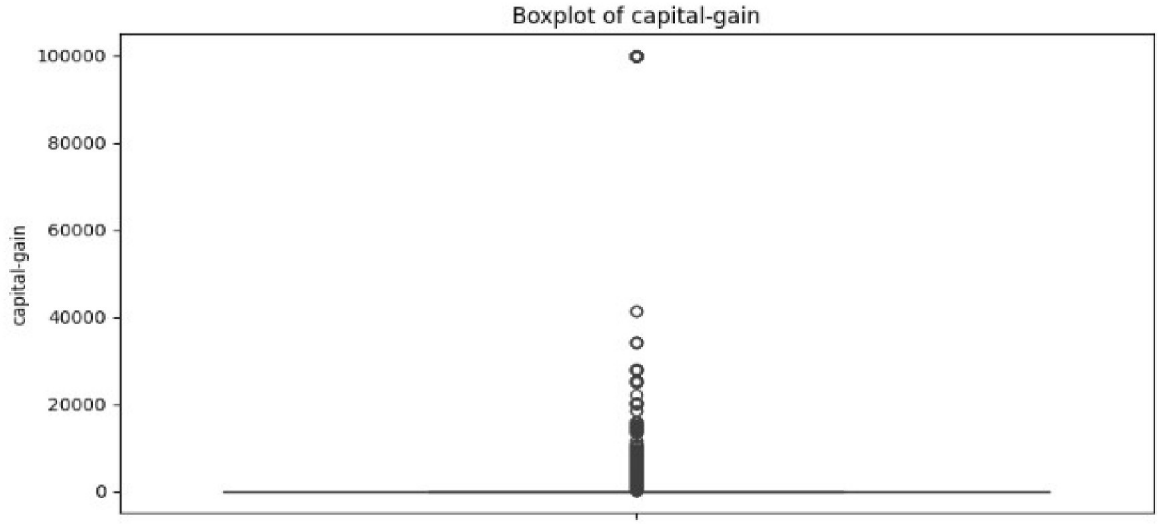

Distribution of annual_income
annual_income
<=50K
>50K
0
5000
10000
15000
20000
25000
count

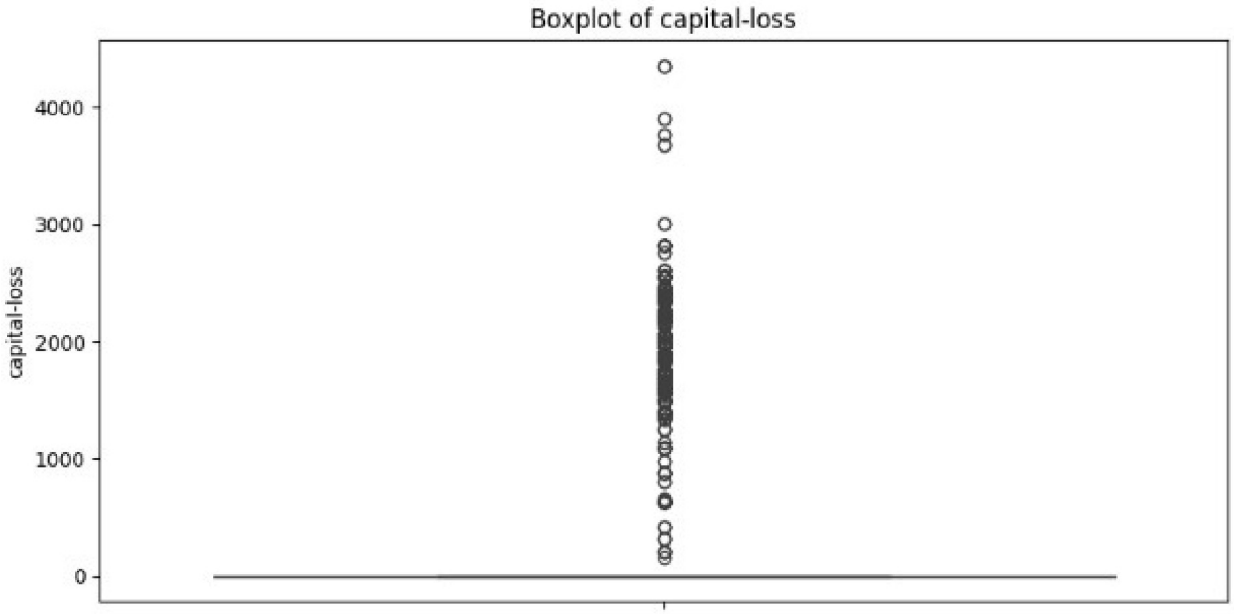

Correlation Heatmap
age
fnlwgt
education-num
capital-gain
capital-loss
hours-per-week
1.00 -0.08 0.04 0.08 0.06 0.07
-0.08 1.00 -0.04 0.00 -0.01 -0.02
0.04 -0.04 1.00 0.12 0.08 0.15
0.08 0.00 0.12 1.00 -0.03 0.08
0.06 -0.01 0.08 -0.03 1.00 0.05
0.07 -0.02 0.15 0.08 0.05 1.00
age
fnlwgt
education-num
capital-gain
capital-loss
hours-per-week
1.0
0.8
0.6
0.4
0.2
0.0

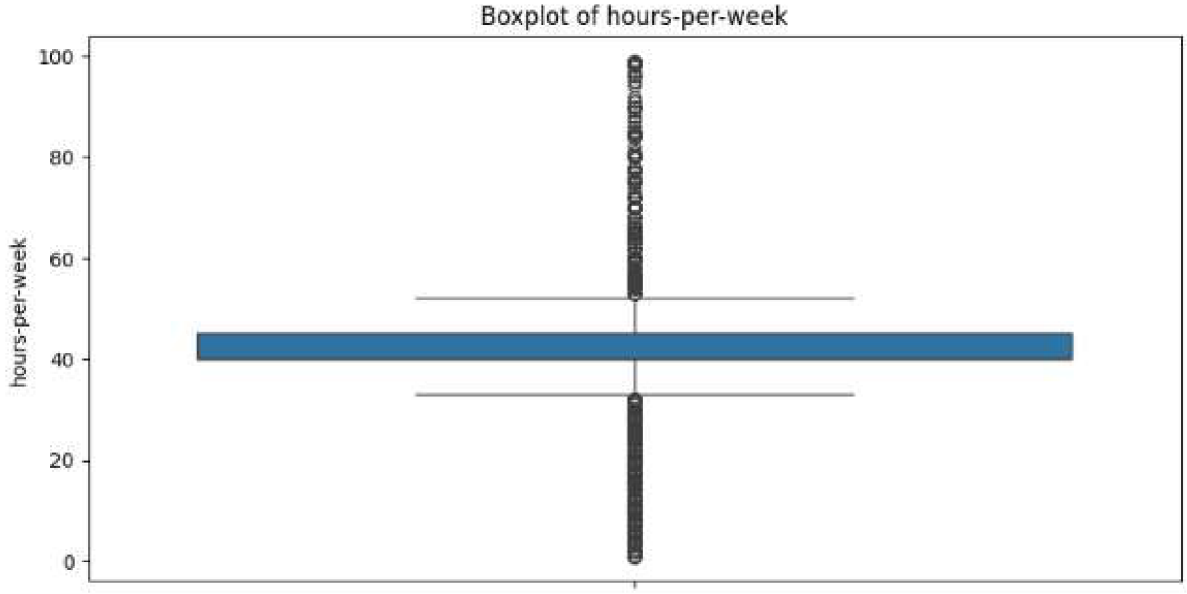

Boxplot of age
age
90
80
70
60
50
40
30
20


Boxplot of fnlwgt
1e6
fnlwgt
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0


Boxplot of education-num
education-num
16
14
12
10
8
6
4
2


Boxplot of capital-gain
capital-gain
100000
80000
60000
40000
20000
0


Boxplot of capital-loss
capital-loss
4000
3000
2000
1000
0


Boxplot of hours-per-week
hours-per-week
100
80
60
40
20
0

2. Use various machine learning algorithms to predict the response variable. Code:-

```python
# Importing required libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import classification_report, roc_auc_score


# Display dataset info and preview the data
print("Dataset Info:")
print(data.info())  # Provides an overview of the dataset
print("\nFirst 5 Rows of the Dataset:")
print(data.head())  # Displays the first few rows to verify correct loading

# Check for the target column
target_column = 'annual_income'  # Adjust this if the target column name is different
if target_column not in data.columns:
    raise ValueError(f"Target column '{target_column}' not found. Available columns: {data.columns.tolist()}")

# Handle missing values (if any)
# Drop rows with missing values (or use other imputation methods as needed)
data = data.dropna()

# Encode categorical variables
# Assuming categorical variables are already encoded. If not, use LabelEncoder or OneHotEncoder.

# Split the data into features (X) and target (y)
X = data.drop(target_column, axis=1)  # Features are all columns except the target
y = data[target_column]  # Target is the 'annual_income' column

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Logistic Regression
print("\nLogistic Regression Performance:")
log_reg = LogisticRegression(max_iter=1000)  # Create the Logistic Regression model
log_reg.fit(X_train, y_train)  # Train the model
y_pred = log_reg.predict(X_test)  # Make predictions
print(classification_report(y_test, y_pred))  # Display the performance metrics
```

```python
# Random Forest Classifier
print("\nRandom Forest Performance:")
rf = RandomForestClassifier(random_state=42)  # Create the Random Forest model
rf.fit(X_train, y_train)  # Train the model
y_pred = rf.predict(X_test)  # Make predictions
print(classification_report(y_test, y_pred))  # Display the performance metrics

# Calculate ROC-AUC for Random Forest
y_prob = rf.predict_proba(X_test)[:, 1]  # Get predicted probabilities for the positive class
roc_auc = roc_auc_score(y_test, y_prob)  # Calculate ROC-AUC score
print(f"ROC-AUC Score for Random Forest: {roc_auc:.2f}")

# Gradient Boosting Classifier
print("\nGradient Boosting Performance:")
gb = GradientBoostingClassifier(random_state=42)  # Create the Gradient Boosting model
gb.fit(X_train, y_train)  # Train the model
y_pred = gb.predict(X_test)  # Make predictions
print(classification_report(y_test, y_pred))  # Display the performance metrics
```

Ans:-

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
Index: 30162 entries, 0 to 32560
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             30162 non-null  int64
 1   workclass       30162 non-null  int64
 2   fnlwgt          30162 non-null  int64
 3   education       30162 non-null  int64
 4   education-num   30162 non-null  int64
 5   marital-status  30162 non-null  int64
 6   occupation      30162 non-null  int64
 7   relationship    30162 non-null  int64
 8   race            30162 non-null  int64
 9   sex             30162 non-null  int64
 10  capital-gain    30162 non-null  int64
 11  capital-loss    30162 non-null  int64
 12  hours-per-week  30162 non-null  int64
 13  native-country  30162 non-null  int64
 14  annual_income   30162 non-null  int64
dtypes: int64(15)
memory usage: 3.7 MB
None

First 5 Rows of the Dataset:
   age  workclass  fnlwgt  education  education-num  marital-status  \
0   39          5   77516          9             13               4
1   50          4   83311          9             13               2
2   38          2  215646         11              9               0
3   53          2  234721          1              7               2
4   28          2  338409          9             13               2
```

```
   occupation  relationship  race  sex  capital-gain  capital-loss  \
0           0             1     4    1          2174             0
1           3             0     4    1             0             0
2           5             1     4    1             0             0
3           5             0     2    1             0             0
4           9             5     2    0             0             0

   hours-per-week  native-country  annual_income
0              40              38              0
1              13              38              0
2              40              38              0
3              40              38              0
4              40               4              0

Logistic Regression Performance:
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
              precision    recall  f1-score   support

           0       0.81      0.95      0.88      4503
           1       0.71      0.35      0.47      1530

    accuracy                           0.80      6033
   macro avg       0.76      0.65      0.67      6033
weighted avg       0.79      0.80      0.77      6033
```

Decision Tree Performance:

	precision	recall	f1-score	support
0	0.87	0.87	0.87	4503
1	0.62	0.62	0.62	1530
accuracy			0.81	6033
macro avg	0.75	0.75	0.75	6033
weighted avg	0.81	0.81	0.81	6033

Random Forest Performance:

	precision	recall	f1-score	support
0	0.89	0.93	0.91	4503
1	0.75	0.65	0.69	1530
accuracy			0.86	6033
macro avg	0.82	0.79	0.80	6033
weighted avg	0.85	0.86	0.85	6033

ROC-AUC Score for Random Forest: 0.91

Gradient Boosting Performance:

	precision	recall	f1-score	support
0	0.88	0.95	0.91	4503
1	0.80	0.61	0.69	1530
accuracy			0.86	6033
macro avg	0.84	0.78	0.80	6033
weighted avg	0.86	0.86	0.86	6033